

OPERATING SYSTEMS AND COMPUTER NETWORKS

THREADS

Lecturer: Milos Antonijevic

THREADS

- Instead of classic view of a single point of execution within a program, a multi-threaded program has more than one point of execution. In this context, each thread is very much like a separate process, except that they share the same address space and can access the same data.
- The context switch between threads is quite similar to the context switch between processes, as the register state of T1 must be saved and the register state of T2 restored before running T2. With processes, we saved state to a **process control block** (PCB) and with threads we use **thread control blocks** (TCBs) to store the state of each thread of a process.
- There are two major advantages when using threads inside a program. We enable parallelism (when each processor does a portion of the work), and prevent blocking program progress due to slow I/O.

CREATING THREADS - EXAMPLE

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("main: begin\n");
    pthread_create(&p1, NULL, mythread, "A");
    pthread_create(&p2, NULL, mythread, "B");
    // join waits for the threads to finish
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("main: end\n");
    return 0;
}
```

```
student@linux$ gcc -pthread threads1.c
```

PASSING ARGUMENTS TO THREAD

```
#include <assert.h>
#include <stdio.h>
#include <pthread.h>

typedef struct {
    int a;
    int b;
} myarg_t;

void *mythread(void *arg) {
    myarg_t *args = (myarg_t *) arg;
    printf("%d %d\n", args->a, args->b);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    myarg_t args = { 10, 20 };
    int rc = pthread_create(&p, NULL, mythread, &args);
    // assert(rc == 0);
    pthread_join(p, NULL);
    printf("done\n");
    return 0;
}
```

GETTING THREAD'S RETURN VALUE

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

typedef struct {
    int x;
    int y;
} myret_t;

void *mythread(void *arg) {
    myret_t *rvals = malloc(sizeof(myret_t));
    rvals->x = 1;
    rvals->y = 2;
    return (void *) rvals;
}

int main(int argc, char *argv[]) {
    pthread_t p;
    myret_t *rvals;
    pthread_create(&p, NULL, mythread, &args);
    pthread_join(p, (void **) &rvals);
    printf("returned %d %d\n", rvals->x, rvals->y);
    free(rvals);
    return 0;
}
```

USING SHARED GLOBAL VARIABLE - EXAMPLE

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
int counter = 0;
int loops;
void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) counter++;
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: threads <loops>\n");
        exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);
    pthread_create(&p1, NULL, worker, NULL);
    pthread_create(&p2, NULL, worker, NULL);
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("main: done\n [counter: %d]\n [should: %d]\n", counter, loops*2);
    return 0;
}
```

This is called **race condition**

BASIC THERMINOLOGY

- **Critical section** is a piece of code that accesses a shared resource, usually a variable or data structure.
- **Race condition** is a state when a multiple threads of execution enter the critical section at roughly the same time; both attempt to update the shared data structure, leading to a indeterministic outcome.
- **Atomic operations** are expressed with the phrase “all or nothing” - either all of them should be executed or none of them, with no in-between state that can be observed.
- Mutual **exclusion primitives** (locks) are used to guarantee that only a single thread will ever enter a critical section, thus avoiding race conditions.

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_lock(&lock);  
pthread_mutex_unlock(&lock);
```

SYSTEM LOCKS

- One of the earliest solutions used to provide mutual exclusion was to disable interrupts for critical sections and ensuring that the critical section will not be interrupted, and thus will execute as if it was atomic.
- Problems:
 - This approach requires that any calling thread can perform a privileged operation (turning interrupts on and off), and thus trust that this facility is not abused (what if some malicious program turns off interrupts and goes into endless loop).
 - The approach does not work on multiprocessors.
 - Turning off interrupts for extended periods of time can lead to interrupts becoming lost. For example, if the CPU missed the fact that a disk device has finished a read request, the OS can will not wake the process waiting for that read.
 - Inefficient approach on modern CPUs.
- To overcome these problem, the hardware support for locking was developed.

HARDWARE LOCKS

- Test-and-set **atomic** instruction:

```
int TestAndSet(int *old_ptr, int new) {  
    int old = *old_ptr; // fetch old value at old_ptr  
    *old_ptr = new; // store 'new' into old_ptr  
    return old; // return the old value  
}
```

- Lock implementation:

```
typedef struct __lock_t {  
    int flag;  
} lock_t;  
  
void init(lock_t *lock) {  
    // 0: lock is available, 1: lock is held  
    lock->flag = 0;  
}  
  
void lock(lock_t *lock) {  
    while (TestAndSet(&lock->flag, 1) == 1) // spin-wait (do nothing)  
    }  
  
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

CONDITION VARIABLES

- In some cases a thread wishes to check whether some condition is true before continuing its execution. For example, a parent thread might wish to check whether a child thread has completed before continuing (like `pthread_join()`).
- One way to implement this is by using shared (global) variable.

```
int done = 0;
void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL); // create child
    while (done == 0) ; // spin
    printf("parent: end\n");
    return 0;
}
```

CONDITION VARIABLES

- The more efficient way of implementation is by using **condition variable**.
- Condition variable is a queue that threads can put themselves on when some state of execution is not as desired (when they wait for something). Some other thread can change that state and then wake up a waiting threads and thus allow them to continue (by signaling on the condition).
- Thread can use **wait()** system call to put itself to sleep, waiting for some condition. The other thread can use **signal()** call when it changed the state of the condition and wants to wake a sleeping thread (see *condition_var.c* code).
- There are two cases that can occur regarding the order of parent and child thread. The first case is that parent continues running itself after creating the child and then calls the code for putting itself to sleep. When the child runs it changes the condition and wakes up the parent.
- The other case is that parent creates the child thread and child thread continues to run. It tries to wake the parent that is not yet gone sleeping.

SEMAPHORES

- **Semaphore** is a synchronization primitive that replaces both locks and condition variables.
- Semaphores can be manipulated by two routines – **sem_wait()** and **sem_post()**.

```
int sem_wait(sem_t *s) {  
    //decrement the value of semaphore by one  
    //wait if value of semaphore is negative  
}
```

```
int sem_post(sem_t *s) {  
    //increment the value of semaphore by one  
    //if there are one or more threads waiting, wake one  
}
```

USE SEMAPHORES AS LOCKS – EXAMPLE

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

sem_t mutex;
int counter = 0;

void *child(void *arg) {
    int i;
    sem_wait(&mutex);
    for (i = 0; i < 10000000; i++) {
        counter++;
    }
    sem_post(&mutex);
    return NULL;
}

int main(int argc, char *argv[]) {
    sem_init(&mutex, 0, 1);
    pthread_t c1, c2;
    pthread_create(&c1, NULL, child, NULL);
    pthread_create(&c2, NULL, child, NULL);
    pthread_join(c1, NULL);
    pthread_join(c2, NULL);
    printf("result: %d (should be 20000000)\n", counter);
    return 0;
}
```

USE SEMAPHORES AS CONDITION VARIABLES

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

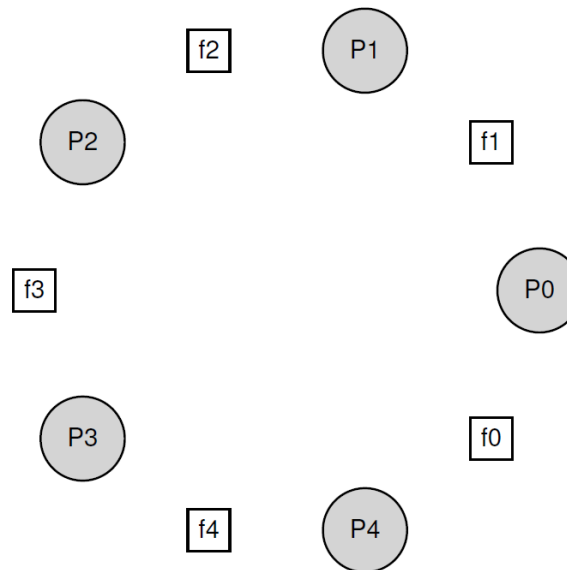
sem_t s;

void *child(void *arg) {
    printf("child\n");
    sem_post(&s); // signal here: child is done
    return NULL;
}

int main(int argc, char *argv[]) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s); // wait here for child
    printf("parent: end\n");
    return 0;
}
```

DEADLOCK – THE EXAMPLE OF DINING PHILOSOPHERS

- The basic setup for the dining philosopher's problem is this: assume there are five philosophers sitting around a table. Between each pair of philosophers is a single fork (five forks in total). The philosophers each have times where they think, and don't need any forks, and times where they eat. In order to eat, a philosopher needs two forks, both the one on their left and the one on their right.



DEADLOCK – THE EXAMPLE OF DINING PHILOSOPHERS

- Pseudo-code for the dining philosopher's case:

```
while (1) {  
    think();  
    get_forks(p);  
    eat();  
    put_forks(p);  
}
```

- Pseudo-code for getting the forks:

```
int left(int p) { return p; }  
int right(int p) { return (p + 1) % 5; }
```


DEADLOCK – THE EXAMPLE OF DINING PHILOSOPHERS

- To acquire the forks, one can grab a "lock" on each one: first the one on the left, and then the one on the right. When the eating is done, they are released.

- Using the semaphores:

```
void get_forks(int p) {  
    sem_wait(&forks[left(p)]);  
    sem_wait(&forks[right(p)]);  
}  
  
void put_forks(int p) {  
    sem_post(&forks[left(p)]);  
    sem_post(&forks[right(p)]);  
}
```

- We have a **deadlock** problem If each philosopher happens to grab the fork on their left before any philosopher can grab the fork on their right, each will be stuck holding one fork and waiting for another, forever.

DEADLOCK SOLUTION ON THE EXAMPLE OF DINING PHILOSOPHERS

- The simplest way to attack this problem is to change how forks are acquired by at least one of the philosophers. Let's assume that philosopher 4 (the highest numbered one) gets the forks in a different order than the others.

```
void get_forks(int p) {  
    if (p == 4) {  
        sem_wait(&forks[right(p)]);  
        sem_wait(&forks[left(p)]);  
    }  
    else {  
        sem_wait(&forks[left(p)]);  
        sem_wait(&forks[right(p)]);  
    }  
}
```